

Introduction to Data Engineering

Course 1, Week 1 - Lecture Notes

Based on Video Lecture Transcripts

Contents

1	Data Engineering Defined	4
1.1	Evolution of Data Engineering	4
1.2	Formal Definition	4
1.3	The Data Engineering Lifecycle	4
1.4	End Use Cases	5
1.5	Data Pipelines	5
1.6	Undercurrents of the Data Engineering Lifecycle	5
2	A Brief History of Data Engineering	5
2.1	The Origins of Digital Data	5
2.2	Timeline of Key Developments	6
2.2.1	1960s - The Beginning	6
2.2.2	1970s - Relational Databases	6
2.2.3	1980s - Data Warehousing	6
2.2.4	1990s - Business Intelligence Era	6
2.2.5	Early 2000s - Big Data Era Begins	6
2.2.6	2004 - The Big Bang Moment	7
2.2.7	The Cloud Revolution	7
2.2.8	Late 2000s and Early 2010s	7
2.3	The Modern Era: Beyond Big Data	7
2.4	The 2010s to Present	8
3	The Data Engineer Among Other Stakeholders	8
3.1	Core Responsibility	8
3.2	Downstream Stakeholders	8
3.2.1	Example: Serving a Business Analyst	9
3.3	Upstream Stakeholders	9
3.3.1	Key Communication Points	9
3.3.2	Benefits of Good Relationships	9
3.4	Summary	10
4	Business Value	10
4.1	Success as a Data Engineer	10
4.2	Advice from Bill Inmon	10
4.3	What Does Business Value Mean?	10
4.4	Forms of Business Value	10
4.5	Prioritization Challenge	11
4.6	Moving Forward	11

5	System Requirements	11
5.1	Delivering Value Through Requirements	11
5.2	Types of Requirements	11
5.2.1	Business Requirements	11
5.2.2	Stakeholder Requirements	11
5.2.3	System Requirements	11
5.3	Scope of System Requirements	12
5.4	The Requirements Gathering Process	12
6	Translate Stakeholder Needs into Specific Requirements	12
6.1	Key Elements of Requirements Gathering	12
6.2	Example: Conversation with a Data Scientist	13
6.2.1	Existing Systems and Pain Points	13
6.2.2	Data Cleaning and Processing	13
6.2.3	Understanding "Real-Time"	13
6.2.4	Example: Marketing Team Actions	14
6.3	Summary of Requirements Gathering	14
7	Thinking Like a Data Engineer	15
7.1	Framework Overview	15
7.2	Stage 1: Determine Business Goals and Stakeholder Needs	15
7.3	Stage 2: Define Functional and Non-Functional Requirements	15
7.4	Stage 3: Choose Tools and Technologies	15
7.4.1	Cost-Benefit Analysis Considerations	16
7.4.2	Prototype and Test	16
7.5	Stage 4: Build, Deploy, and Iterate	16
7.6	The Cyclical Nature of Data Engineering	16
7.7	Framework Application in This Course	17
8	Data Engineering on the Cloud	17
8.1	High-Level Mental Framework	17
8.2	Variability in Tools and Technologies	17
8.3	Cloud vs. On-Premises	17
8.3.1	Historical Context	17
8.3.2	Current Landscape	17
8.4	Cloud Providers	18
8.4.1	AWS - The Pioneer	18
8.4.2	Other Providers	18
8.5	Approach of This Specialization	18
8.6	Learning Approach	18
8.7	Course Outcome	19
9	Introduction to the AWS Cloud	19
9.1	Defining Cloud Computing	19
9.2	IT Resources on AWS	19
9.2.1	Compute Resources	19
9.2.2	Storage Resources	19
9.2.3	Networking Resources	20
9.2.4	Beyond Core Categories	20
9.3	Advantages of Cloud Computing	20
9.4	AWS Global Infrastructure	20
9.4.1	Physical Data Centers	20

9.4.2	Regions	21
9.4.3	Availability Zones (AZs)	21
9.4.4	AWS Global Network	21
9.5	Working with AWS Resources	21
10	Introduction to AWS Core Services	22
10.1	Overview	22
10.2	Compute Services	22
10.2.1	Amazon Elastic Compute Cloud (EC2)	22
10.2.2	Other Compute Options	23
10.3	Networking Services	23
10.3.1	Amazon Virtual Private Cloud (VPC)	23
10.4	Storage Services	23
10.4.1	Object Storage	24
10.4.2	Block Storage	24
10.4.3	File Storage	24
10.5	Database Services	25
10.5.1	Amazon Relational Database Service (RDS)	25
10.5.2	Amazon Redshift	25
10.6	Security	25
10.6.1	The Shared Responsibility Model	25
10.6.2	Variation Across Services	26
10.7	Next Steps	27
11	Summary and Key Takeaways	27
11.1	Week 1 Overview	27
11.2	Moving Forward	28

1 Data Engineering Defined

1.1 Evolution of Data Engineering

The field of data engineering has undergone significant transformation over time. Originally, data engineers were software engineers whose primary work focused on building software applications for organizations. In the early days, data generated by software applications was largely regarded as a **byproduct or exhaust**—useful primarily for troubleshooting or monitoring application health, but not considered to have much intrinsic value on its own.

Over time, as organizations began to recognize the intrinsic value of data, and as the volume and variety of data continued to grow from multiple sources (software applications, user uploads, digitized records), software engineers became increasingly focused on building systems specifically for:

- Ingesting data
- Storing data
- Transforming data
- Serving data for various use cases

With the emergence of data engineering as a central function within data-driven organizations, the role of the **data engineer** was born.

1.2 Formal Definition

According to the book *Fundamentals of Data Engineering*, data engineering is defined as:

Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning.

Data engineering sits at the intersection of:

- Security
- Data Management
- DataOps
- Data Architecture
- Orchestration
- Software Engineering

1.3 The Data Engineering Lifecycle

The data engineering lifecycle comprises a series of stages:

1. **Generation** - Data generation and source systems (applications, user-generated data, sensor measurements)
2. **Ingestion** - Getting data from source systems
3. **Transformation** - Processing and transforming raw data

4. **Storage** - Storing data (spans across all stages)
5. **Serving** - Making data available for downstream use

The ingestion, transformation, storage, and serving stages represent the core focus areas for data engineers. Storage is particularly important as it's an integral part of each of the other stages.

1.4 End Use Cases

The data engineering lifecycle serves several end use cases:

- **Analytics** - Data analysis and business intelligence
- **Machine Learning** - Training and deploying ML models
- **Reverse ETL** - Sending transformed data back to source systems

1.5 Data Pipelines

A **data pipeline** is a combination of architecture, systems, and processes that move data through the stages of the data engineering lifecycle. The pipeline begins with getting data from source systems and ends with serving data for use cases including analytics and machine learning.

In simpler terms: A data engineer's job is to get raw data from somewhere, turn it into something useful, and then make it available for downstream use cases.

1.6 Undercurrents of the Data Engineering Lifecycle

The six **undercurrents** are not stages of the lifecycle, but rather they span the entire lifecycle:

1. Security
2. Data Management
3. DataOps
4. Data Architecture
5. Orchestration
6. Software Engineering

Each undercurrent is relevant to all stages of the lifecycle.

2 A Brief History of Data Engineering

2.1 The Origins of Digital Data

Data exists everywhere and comprises the building blocks of information. While data has existed in various forms since the beginning of time, the focus in data engineering is on **digitally recorded data**—data that can be stored on a computer or sent through the Internet.

2.2 Timeline of Key Developments

2.2.1 1960s - The Beginning

- Advent of computers
- First computerized databases were introduced

2.2.2 1970s - Relational Databases

- Relational databases emerged
- Engineers at IBM developed **Structured Query Language (SQL)**

2.2.3 1980s - Data Warehousing

- Bill Inmon developed the first **data warehouse**
- Purpose: transforming data to support analytical decision-making

2.2.4 1990s - Business Intelligence Era

- Data systems grew, requiring dedicated tools and data pipelines for reporting and business intelligence
- Ralph Kimball and Bill Inmon developed their respective data modeling approaches for analytics
- Internet went mainstream (mid-1990s)
- Emergence of web-first companies (e.g., Amazon)
- Dotcom boom led to rapid growth in web applications and backend systems (servers, databases, storage)

2.2.5 Early 2000s - Big Data Era Begins

- Dotcom bust survivors (Yahoo, Google, Amazon) grew into powerful tech companies
- Traditional relational databases and data warehouses could not handle the explosion of data
- **Big Data** era began

Big Data Definition: Extremely large datasets that may be analyzed computationally to reveal patterns, trends, and associations, especially relating to human behavior and interactions.

Three Vs of Big Data:

1. **Velocity** - High speed of data generation
2. **Variety** - Wide variety of data types
3. **Volume** - Large amounts of data

2.2.6 2004 - The Big Bang Moment

- Google published a paper on **MapReduce**, an ultra-scalable data processing paradigm
- This publication constituted a "big bang moment" for data technologies
- Inspired engineers at Yahoo to develop **Apache Hadoop** (2006)

Hadoop's impact cannot be overstated—it drew software engineers interested in large-scale data problems to this new open-source technology ecosystem. As companies saw their data grow into terabytes and petabytes, the era of the big data engineer was born.

2.2.7 The Cloud Revolution

Around the same time, Amazon created:

- **Amazon Elastic Cloud Compute (EC2)** - Scalable and flexible computing environment
- **Amazon Simple Storage Service (S3)** - Infinitely scalable storage
- **Amazon DynamoDB** - Highly scalable NoSQL database

Amazon offered these services through **Amazon Web Services (AWS)**, becoming the first popular public cloud. AWS provided an ultra-flexible, pay-as-you-go resource marketplace. Other public clouds soon followed, including Google Cloud Platform and Microsoft Azure.

The public cloud is arguably one of the most significant innovations of the 21st century, spawning a revolution in how software and data applications are developed and deployed.

2.2.8 Late 2000s and Early 2010s

- Small startups gained access to the same bleeding-edge data tools used by top tech companies
- Transition from **batch computing** (processing data in chunks) to **event streaming** (continuous flow of individual events)
- Era of big, real-time data began

2.3 The Modern Era: Beyond Big Data

Despite previous popularity, the term "big data" has lost momentum. While open-source big data tools were powerful, managing them required excessive time and resources. Companies often employed entire teams of big data engineers, costing millions of dollars to maintain these systems, with arguably less time spent delivering business insights and value.

Today's Reality:

- Data moves faster than ever and continues to grow larger
- Big data processing has become so accessible it no longer merits a separate term
- Every company aims to work with and derive value from its data, regardless of actual data size
- Big data engineers are now simply **data engineers**

2.4 The 2010s to Present

- Emergence of cloud-first, open-source, and third-party products
- Working with data at scale became far simpler than in the big data era
- Data sources and formats continue to grow in variety and size
- Data engineering is increasingly a discipline of **interoperation**—connecting various technologies like Lego bricks to serve business goals

The Role Today: The data engineer role is further up the value chain than ever before and continues to rise. Data engineers now have the opportunity to:

- Stand on the shoulders of giants
- Build powerful, scalable data systems using proven tools and technologies
- Contribute to the development of tools and technologies
- Build the data solutions of tomorrow

Building robust data systems is now at the center of business strategy across industries, allowing data engineers to directly contribute to achieving business goals and delivering value.

3 The Data Engineer Among Other Stakeholders

3.1 Core Responsibility

A data engineer's job is to get raw data from somewhere, turn it into something useful, and make it available for downstream use cases. This cannot happen in a vacuum—understanding the needs of downstream consumers is essential for success.

3.2 Downstream Stakeholders

Who are they?

- Business analysts
- Data scientists
- Machine learning engineers
- Salespeople
- Product managers
- Marketing professionals
- Executives

3.2.1 Example: Serving a Business Analyst

Suppose a downstream data consumer is a business analyst who needs to run SQL queries against a database for various analyses. To serve them successfully, discuss:

- **Query frequency** - How often do they need to refresh dashboards?
- **Data requirements** - What information needs to be retrieved?
- **Pre-processing** - Would certain joins or aggregations help speed up queries?
- **Latency tolerance** - Is hour-old or day-old data acceptable, or do they need real-time data?
- **Data definitions** - Align on specific metrics (e.g., time zones for daily sales totals)

3.3 Upstream Stakeholders

Who are they?

- Software engineers who build source systems
- Third-party developers responsible for external data sources

In this relationship, the data engineer is the consumer, and the source system owners are serving them.

3.3.1 Key Communication Points

Communicate with source system owners to understand:

- **Volume** - How much data to expect
- **Frequency** - How often data is generated
- **Format** - Data structure and schema
- **Security and compliance** - Data security and regulatory requirements
- **Changes and disruptions** - Advance notice of outages or schema changes

3.3.2 Benefits of Good Relationships

With open lines of communication, upstream stakeholders can:

- Inform about expected disruptions in advance
- Notify about schema changes
- Allow influence on how raw data is served

Even when source systems are external and out of your control, connecting with system owners helps develop a better understanding of the applications that generate the data you consume.

3.4 Summary

Data engineers have stakeholders both downstream and upstream:

- **Upstream:** Connect with source system owners to understand ingested data and potential disruptions
- **Downstream:** Understand how served data adds value for the organization and relates to stakeholder goals

4 Business Value

4.1 Success as a Data Engineer

Any company that hires a data engineer does so because they believe the engineer will add value for the organization. Understanding and delivering business value is crucial for success.

4.2 Advice from Bill Inmon

Bill Inmon, a thought leader in the field of data, emphasizes:

”Go to where the money is. If you want to have long-term, great success in our industry, find business value. Don’t get hung up on every technology that comes out. Business value drives everything we do in technology. If you don’t care about success and want to do fancy stuff that looks interesting, go ahead—but don’t expect to have any great reward for what you’ve done.”

4.3 What Does Business Value Mean?

Business value is somewhat ”in the eye of the beholder”—it depends on perception.

Examples:

- If leadership perceives your work as contributing to revenue growth, you’ll be recognized as adding value
- If your work is perceived as costing money with little ROI, your position may be at risk

However, business value is not just profit and loss. It takes many forms and is often determined by stakeholders themselves.

4.4 Forms of Business Value

Business value can come in various forms:

- Increased revenue
- Cost savings
- Work made more efficient
- Helping with successful product launches
- Supporting better decision-making

Key Principle: When fulfilling stakeholder needs, you’re providing them with value. The amount of value is determined by those to whom you are providing that value.

4.5 Prioritization Challenge

Often, stakeholder needs far exceed available capacity and resources. Data engineers must:

- Prioritize projects
- Determine feasibility
- Estimate implementation time
- Balance competing demands

4.6 Moving Forward

The next step after identifying business value is understanding system requirements and translating stakeholder needs into concrete technical specifications.

5 System Requirements

5.1 Delivering Value Through Requirements

To ensure you're delivering value for stakeholders:

1. Understand their needs
2. Translate those needs into requirements for the systems you build

5.2 Types of Requirements

The term "requirements" can mean different things in business and engineering contexts.

5.2.1 Business Requirements

- Define high-level goals of the business
- Examples: Growing revenue, increasing user base

5.2.2 Stakeholder Requirements

- Needs of individuals within the organization
- Things they need to get their job done well

5.2.3 System Requirements

- Describe what a system needs to be able to do
- Must meet business and stakeholder requirements
- Split into two categories:

Functional Requirements (The "What") Things the system needs to be able to do.
Examples in data engineering:

- Providing regular updates to a database serving analytics dashboards
- Alerting users when there's an anomaly in the data

Non-Functional Requirements (The "How") How the system will accomplish what it needs to do.

Examples:

- Technical specifications for ingestion
- Orchestration approaches
- Storage solutions used within data pipelines

5.3 Scope of System Requirements

System requirements can include:

- High-level business and stakeholder requirements
- Features and attributes of data products
- Memory and storage capacity specifications
- Computation and database resources
- Cost constraints
- Security and regulatory requirements

5.4 The Requirements Gathering Process

First and Most Important Step: Gather requirements for your system.

The Challenge: Stakeholders typically don't speak in terms of concrete system requirements. Instead, they think about business goals. It's the data engineer's job to translate business needs into system requirements.

Key Approach: The process always starts with having conversations with stakeholders. How these conversations unfold depends on:

- The stakeholder's technical level regarding data and data systems
- Their role within the organization

6 Translate Stakeholder Needs into Specific Requirements

6.1 Key Elements of Requirements Gathering

Any requirements gathering effort should include the following elements:

1. Learn about existing systems

- What solutions are currently in place?
- What are the pain points or problems with those systems?

2. Learn about planned actions

- What actions will stakeholders take based on the data you serve?

3. Confirm understanding

- Repeat what you learned back to stakeholders

- Verify that you understood everything correctly

4. Identify other stakeholders

- Determine if you need to talk to additional people
- Fill in any missing information about existing systems or planned actions

6.2 Example: Conversation with a Data Scientist

6.2.1 Existing Systems and Pain Points

Scenario: The marketing team needs real-time analysis of product sales by region, but they're only getting a daily data dump from the software team to avoid compromising the production database.

Key Insights:

- Not using production databases for analytics is generally good practice
- Running complex queries against a production database while writing new information can bring the system down
- Problems occur due to schema changes or other anomalies in the data

Action Items:

- Build automatic checks on ingested data to ensure expectations are met
- Communicate directly with source system owners for advance notice of changes
- Talk to software engineers who maintain the source system and database
- Understand what ingestion solutions are better than daily data dumps
- Learn what disruptions or changes to expect and how to get advance notice

6.2.2 Data Cleaning and Processing

Need: Automate the ingestion and transformation of data into the required format.

Requirements Identified:

- **Functional Requirement:** System needs to ingest, transform, and serve data in the format the data scientist needs
- **Non-Functional Requirement:** Latency requirement—how fast data needs to be available relative to when it was recorded

6.2.3 Understanding "Real-Time"

Problem: People use terms like "real-time" rather loosely.

Examples of "real-time" interpretations:

- Monthly reports
- Daily updates
- Hourly refreshes
- Sub-second latency

Key Tactic: Ask stakeholders what action they plan to take based on the data you serve them.

Important Distinction: Asking "what action they plan to take" is different from asking "what they need."

Why This Matters:

- Stakeholders often translate their needs into functional requirements themselves
- Their translation may not be accurate or complete
- Understanding what they hope to do with the data helps you map their plans to actual needs
- You may discover different or additional functional requirements they haven't identified

6.2.4 Example: Marketing Team Actions

Planned Actions:

1. Making decisions about marketing campaigns based on analytics dashboards
2. Serving product recommendations in the platform

Analysis:

- **Recommender system:** Product recommendations need to be provided in near real-time (seconds or less) to customers browsing the platform
- **Dashboards:** Not yet clear what action the marketing team plans to take—need to talk to the marketing team to clarify what "real-time" means in this context

6.3 Summary of Requirements Gathering

Key Elements:

1. Identify existing systems and pain points
2. Ask what actions stakeholders plan to take with the data
3. Confirm what you've learned
4. Identify additional stakeholders to talk to

Outcome:

- Learn about existing solutions and pain points
- Start to identify system requirements
- Identify additional stakeholders (marketing team, software engineering team)

Once all requirements are gathered, it's time to choose the tools and technologies that will meet those requirements.

7 Thinking Like a Data Engineer

7.1 Framework Overview

Requirements gathering is not unique to data engineering—it appears in various product development and product management scenarios. In data engineering, work includes everything from requirements gathering to implementation of data systems.

This section presents a framework for "thinking like a data engineer" that can be applied to any project.

7.2 Stage 1: Determine Business Goals and Stakeholder Needs

Main Objective: Determine what business goals and stakeholder needs will drive the project.

Steps:

1. Get clear on high-level business goals for the company
2. Identify all stakeholders with needs related to the project
3. Understand how their needs tie into higher-level business goals
4. Have conversations with individual stakeholders
5. Learn what systems are currently in place (building on or replacing)
6. Identify stakeholder needs beyond what current systems provide
7. Ask stakeholders what action they plan to take with the data products provided

Outcome: Zero in on the actual functional requirements of the system to build.

7.3 Stage 2: Define Functional and Non-Functional Requirements

Main Objective: Turn stakeholder needs into functional and non-functional requirements for the system.

Key Tasks:

- **Functional Requirements:** Describe what the system must be able to do to meet stakeholder needs
- **Non-Functional Requirements:** Describe the technical specifications of how the system will accomplish what it needs to do

Final Step:

1. Document conclusions
2. Confirm with stakeholders that a system designed per these requirements will meet their needs

7.4 Stage 3: Choose Tools and Technologies

Main Objective: Select the tools and technologies to build the system.

Process:

1. Identify tools and technologies that can meet system requirements
2. Recognize that multiple tools can typically meet any individual requirement
3. Evaluate trade-offs between options
4. Conduct cost-benefit analysis

7.4.1 Cost-Benefit Analysis Considerations

- Licensing fees
- Estimated spending on cloud resources
- Resources needed to build the system
- Resources needed to maintain the system

7.4.2 Prototype and Test

Important Step: Set up a prototype of the system to:

- Test whether it meets expectations
- Verify it can deliver value for stakeholders

Before investing time and energy into building the full system, testing the design is critical.

Iteration:

- Check back with stakeholders
- Let them evaluate whether the designed system will deliver value
- Spend as much time as needed iterating on the prototype
- Ensure system success before moving to production

7.5 Stage 4: Build, Deploy, and Iterate

Final Steps:

1. Build and deploy the data system
2. Continuously monitor and evaluate system performance
3. Iterate to keep improving where possible

7.6 The Cyclical Nature of Data Engineering

While presented as a sequence of four stages, this framework is actually an **ongoing and cyclical process**.

Why?

- Business goals evolve
- Stakeholder needs change
- Data systems must evolve accordingly
- New tools or technologies may provide advantages

Continuous Responsibilities:

- Always communicate with stakeholders about needs and expectations
- Evaluate needs in terms of system requirements
- Update systems as needed to meet stakeholder needs

7.7 Framework Application in This Course

Week 1: Focus on Stages 1 and 2

- Identifying business goals and stakeholder needs
- Defining requirements

Week 4:

- Continue conversation with data scientist
- Have conversations with other stakeholders
- Define requirements
- Turn requirements into technology specifications and tooling choices

Lab Exercise:

- Complete the process
- Implement a data architecture that meets requirements

8 Data Engineering on the Cloud

8.1 High-Level Mental Framework

This specialization focuses on data engineering with a high-level mental framework for thinking like a data engineer. This framework is important for everything that follows and will be revisited throughout the courses.

8.2 Variability in Tools and Technologies

The actual set of tools and technologies used by data engineers can vary significantly from one company to another. However, with the advent of the public cloud and proliferation of open-source and proprietary tools, companies of all sizes and across all industries now have access to the same high-performance tools and technologies.

8.3 Cloud vs. On-Premises

8.3.1 Historical Context

Not long ago, the only option was for companies to build and maintain their own data infrastructure in-house, known as **on-premises systems**.

8.3.2 Current Landscape

Today:

- Many companies are choosing to build data systems entirely on the cloud
- Some companies still maintain some or all infrastructure on-premises due to:
 - Regulatory constraints
 - Legacy systems they prefer to maintain on-premises

In the wild today:

- May work at a company with some on-premises systems
- May be tasked with migrating an on-premises system to the cloud
- Increasingly likely to work exclusively on the cloud

8.4 Cloud Providers

8.4.1 AWS - The Pioneer

- First popular public cloud
- Remains the most widely used today

8.4.2 Other Providers

- Google Cloud Platform
- Microsoft Azure
- Others

Specific cloud provider depends on where you work.

8.5 Approach of This Specialization

Cloud-First Approach:

- Won't dwell on on-premises system nuances
- Won't focus on migrating from on-premises to cloud
- Will prepare you for scenarios most likely to encounter as a data engineer

Partnership with AWS:

- Courses developed in partnership with AWS
- Lab exercises: Build data pipelines and architectures on AWS Cloud
- Use same tools and technologies used by thousands of companies worldwide

8.6 Learning Approach

Not Comprehensive AWS Training:

- Aim is not to teach everything about data engineering on AWS
- Will take a "just-in-time" approach
- Learn specific tools for each lab directly as encountered

Prerequisites:

- No prerequisite knowledge of cloud computing required
- Broader familiarity with cloud computing basics and terminology will help build context

8.7 Course Outcome

By the end of the first course:

- Able to take technical requirements for a system
- Stand up a data pipeline with appropriate cloud-based tools and technologies
- Meet system requirements using AWS services

9 Introduction to the AWS Cloud

9.1 Defining Cloud Computing

AWS Cloud Definition: The on-demand delivery of IT resources over the Internet with pay-as-you-go pricing.

Key Characteristics:

- Provides resources needed to build data systems or applications essentially instantly
- Resources can be shut down when no longer needed
- Pay only for what you use at the end of each month

Contrast with On-Premises:

- On-premises: Committed to IT resource purchases for years
- Cloud: Flexible, on-demand access with no long-term commitments

9.2 IT Resources on AWS

IT Resources Include:

- Compute
- Storage
- Networking

9.2.1 Compute Resources

Places to run code:

- Virtual machines
- Container hosting services
- Serverless functions

9.2.2 Storage Resources

Places to store data:

- Amazon Simple Storage Service (S3)
- Amazon Elastic Block Store
- Database services:
 - Relational databases
 - NoSQL databases
 - Graph databases

9.2.3 Networking Resources

Connect resources to each other and to the outside Internet:

- **Amazon Virtual Private Cloud (VPC):** Your own private network in the cloud

9.2.4 Beyond Core Categories

Many more categories of AWS resources and services:

- Security
- Data streaming
- Ingestion
- Transformation
- Others

9.3 Advantages of Cloud Computing

Scalability and Elasticity:

Cloud resources are **scalable** and **elastic**.

Example: S3 Object Storage

- Don't need to worry ahead of time about storage capacity
- Don't need to manage scaling operations as dataset grows or shrinks
- Service automatically scales up and down (elastic)

Benefits:

- No need to guess capacity ahead of time
- Always ready for spikes in demand or changing traffic
- More easily avoid paying for unneeded resources

Analogy: Like Electricity

- Only pay for what you use
- Details of how it's made or delivered are abstracted
- At the end of the month, receive a bill for usage

9.4 AWS Global Infrastructure

9.4.1 Physical Data Centers

- AWS resources and services hosted in physical data centers built worldwide
- Services accessed over the Internet through APIs
- Build solutions without needing to build and manage own data centers

9.4.2 Regions

Definition: Collections of data centers within geographical areas where AWS services can be used.

Purpose:

- Ensures reliability and availability of systems
- Allows for resilience in case an individual data center goes offline

Naming Convention:

- Named after geographic area
- Examples: US East Northern Virginia, Asia Pacific Mumbai, Europe Frankfurt

9.4.3 Availability Zones (AZs)

Definition: Smaller groupings of data centers within a region.

Design:

- Far enough apart that localized issues don't affect multiple AZs
- If something happens (electrical outage, flood, AZ-level failure), can fail over to other AZs
- Other AZs absorb traffic so systems remain unaffected

Structure:

- Multiple data centers → One availability zone
- Multiple availability zones → One AWS region

9.4.4 AWS Global Network

- Data centers and availability zones connected with low-latency links
- Run on AWS global network of fiber cables connecting facilities

9.5 Working with AWS Resources

Resource Creation:

- Select the region
- Potentially select the availability zone (depending on the service)
- Behind the scenes: Data centers and AZs connected through the global network

Building Solutions:

- Often combine multiple AWS resources and services together like building blocks
- AWS offers over 200 services
- Some general-purpose, others specific in intended use

As a data engineer:

- Will combine multiple AWS resources and services to form solutions
- More to AWS global infrastructure than covered here
- Concepts covered will come up repeatedly as you learn more about AWS

10 Introduction to AWS Core Services

10.1 Overview

Now that you're familiar with what the AWS cloud is and how it works at a high level, this section introduces core AWS services encountered throughout the courses.

Organization: Five categories of core services

1. Compute
2. Network
3. Storage
4. Databases
5. Security

Note: AWS comprises much more than these five categories, but starting with these core categories makes the rest easier to conceptualize.

10.2 Compute Services

10.2.1 Amazon Elastic Compute Cloud (EC2)

Definition: Service that provides virtual machines (VMs) on AWS.

What is a VM?

- A virtual computer or server
- Can run any operating system: Linux, macOS, Windows
- Can run any applications, just like on a physical computer
- Hosted on AWS cloud

Terminology:

- When you launch a single VM using EC2 → **Amazon EC2 instance**

Control and Flexibility:

- Complete control over the instance
- Control operating system
- Control applications
- Control everything that happens on the instance
- Very flexible option for workloads

Use Cases:

- Development machine for programming
- Web server
- Containers
- Machine learning workloads

- Any other workload

Scalability:

- Deploy a single EC2 instance
- Deploy a fleet of instances
- Scale horizontally to match demand

10.2.2 Other Compute Options

Beyond EC2, there are many compute options on AWS:

- **AWS Lambda:** Serverless functions—host code that runs in response to triggers or events
- **Amazon Elastic Container Service (ECS):** Container hosting service
- **Amazon Elastic Kubernetes Service (EKS):** Container hosting service for Kubernetes

10.3 Networking Services

10.3.1 Amazon Virtual Private Cloud (VPC)

Purpose: Whenever you create an EC2 instance or many other types of AWS resources, you need to place it into a network.

Definition: A private network in the cloud that you can create and control, isolated from other networks in the AWS cloud.

Characteristics:

- Choose the size of the private IP space
- Partition it into smaller networks called **subnets**
- Single VPC spans all availability zones within a region
- Cannot span across regions

Regional Boundaries:

- Most AWS resources are region-bound
- Data and resources don't leave the region unless you specifically build solutions to behave that way
- Important for compliance and security purposes

Resource Placement:

- When creating certain AWS resources (EC2 instances, instance-based databases)
- Select which VPC you want
- Select which AZ to place it in

10.4 Storage Services

On AWS, there are multiple different types of storage: object, block, and file storage.

10.4.1 Object Storage

Amazon S3 (Simple Storage Service):

Use Cases:

- Most often used for storing unstructured data
- Documents, logs, photos, videos
- Can store any kind of data

In this specialization:

- Will spend significant time getting familiar with Amazon S3
- Featured extensively in upcoming labs

10.4.2 Block Storage

Amazon Elastic Block Store (EBS):

Use Cases:

- Database storage
- Virtual machine file systems
- Environments where low latency and high performance are critical

How it Works:

- Attach block storage devices (EBS volumes) to EC2 instances
- Volumes mount to the OS
- Store and access data using programs running on EC2

10.4.3 File Storage

Amazon Elastic File System (EFS):

Characteristics:

- Most familiar type of storage for average non-technical users
- Data organized into files and directories in a hierarchical structure
- Like file system on laptop or shared file system at work

Functionality:

- Managed service providing scalable file storage solution
- Can be mounted to multiple different systems at once for file access

10.5 Database Services

Relationship to Storage:

- Databases are another kind of storage service
- Use block storage behind the scenes to store data
- Provide special functionality for managing structured data

Database Functionality:

- Enabling complex querying
- Data indexing
- Other features not typically offered by general storage services

10.5.1 Amazon Relational Database Service (RDS)

Definition: Cloud-based relational database service.

Relevance:

- As a data engineer, frequently work with tabular data in relational database format
- Will become very familiar with RDS in these courses

10.5.2 Amazon Redshift

Definition: Data warehouse service.

Functionality:

- Store, transform, and serve data for end use cases

In this specialization:

- RDS and Redshift are the two database services you'll see the most
- Other database services exist on AWS and will be introduced later

10.6 Security

10.6.1 The Shared Responsibility Model

Definition: AWS follows a shared responsibility model for cloud security.

Principle:

- **AWS:** Responsible for security *of* the cloud
- **You:** Responsible for security *in* the cloud

Analogy: High-Rise Apartment Building Building Management (AWS):

- Ensuring physical building is safe
- Building is up to code
- Each apartment has working locks
- Maybe other security measures

Tenant (You):

- Responsible for using security features
- Making sure door is locked when you leave
- Making sure door is locked at night

Shared Responsibility: Takes both building management and tenant doing their part to ensure any given apartment is secure.

Example: Amazon EC2 AWS Responsibility (Security *of* the Cloud):

- Operating and managing underlying components
- Physical hardware and facilities where EC2 instance runs
- Up through the hypervisor layer (where compute resources are provisioned)
- 100% responsibility for security of these components

Your Responsibility (Security *in* the Cloud):

- Management of guest operating system for the VM
- Software updates and patching
- Setting up networking configurations (firewalls)
- Managing access to your data
- Using proper encryption techniques when needed
- 100% responsibility for security of these aspects

Result: Takes both you and AWS to maintain a secure EC2 instance, and the same goes for any other resource on the cloud.

10.6.2 Variation Across Services

- Each AWS service works differently
- Draws the line in a different place for where AWS's responsibility ends and yours begins
- Will learn more about this concept as you move through the courses and get exposed to different AWS services

10.7 Next Steps

Summary:

- Brief introduction to core services encountered in these courses
- Compute, networking, storage, databases, security

Lab Access:

- All labs in these courses provide access to AWS cloud resources
- No need to create your own account

Optional Exploration:

- Can create a free AWS account if desired
- Follow along to get oriented
- Instructions for setting up AWS account in next reading item
- Totally optional—can be successful in courses without creating own account

11 Summary and Key Takeaways

11.1 Week 1 Overview

This first week covered foundational concepts essential for success as a data engineer:

1. Data Engineering Definition and Lifecycle

- Data engineering transforms raw data into useful information
- The lifecycle includes: generation, ingestion, transformation, storage, and serving
- Six undercurrents span the entire lifecycle

2. Historical Context

- Evolution from 1960s databases to modern cloud-based systems
- The big data era and its transformation
- The cloud revolution and its impact

3. Stakeholder Management

- Understanding downstream and upstream stakeholders
- Communication strategies
- Delivering business value

4. Requirements Engineering

- Types of requirements: business, stakeholder, system (functional/non-functional)
- Requirements gathering process
- Translating stakeholder needs into technical specifications

5. Thinking Like a Data Engineer

- Four-stage framework

- Cyclical nature of data engineering work
- Importance of prototyping and iteration

6. Cloud Computing with AWS

- Cloud computing fundamentals
- AWS global infrastructure (regions, availability zones)
- Core AWS services: compute, network, storage, databases, security
- Shared responsibility model

11.2 Moving Forward

In Week 2:

- Zoom in on each stage of the data engineering lifecycle
- Explore undercurrents in detail
- Understand how all pieces fit together

In Week 4:

- Continue requirements gathering conversations
- Define complete requirements
- Choose appropriate tools and technologies
- Implement data architecture in lab exercise

Key Principle to Remember:

As a data engineer, always focus on delivering business value by understanding stakeholder needs, translating those needs into system requirements, and building scalable, reliable data systems that serve those needs.